

ResearchBench: Recomposing Research Papers into Task Graphs for Evaluating Autonomous AI Research Agents

Annika Kaul Singh
Infinity Inc.
research@infinity.inc

Abstract

The goal of automated AI research is an agent that can take a hypothesis, implement it, run the experiments, and report faithful results—but progress toward this goal is hard to evaluate precisely because novel research has no ground truth. Paper replication offers an escape: the sub-tasks of replicating a published paper are, to a first approximation, the same sub-tasks required to execute novel research, yet the paper’s reported numbers provide a checkable target. PaperBench (2024) made replication gradeable by decomposing papers into expert-authored “rubrics.” What is different in 2026 is that these decompositions—which we generalize to *task graphs*, trees of checkable tasks augmented with dependency edges between task groups—and their executable correctness tests can be *generated automatically* by terminal agents, at unbounded size, for any paper. Task graphs turn the binary notion of replication into a score: the number of sub-tasks an agent successfully implements. We present **ResearchBench**, a public corpus of 100 papers on LLM inference optimization and adjacent systems research, each recomposed into a task graph, organized into nested splits (**ResearchBench-10/-20/-50/-100**) by run maturity, replicability, and compute burden, and browsable as interactive graphs at <https://infinity-ai-institute.github.io/research-bench-rubrics/>. On the run-backed 20-paper split we report agent success rates across three evaluation attempts: a clean baseline (Claude Code closes 5.95%, Codex 1.21% of implementable claims), machine-readable failure attribution locating the bottleneck in *harness legibility*, and a one-paragraph intervention plus raised test-time compute that lifts both agents roughly fivefold with models unchanged. We further describe behavioral defenses against reward hacking that check facts the agent did not author. We argue task graphs are more than an evaluation device: because papers decompose into the same reusable sub-tasks that novel research composes, a library of task graphs is a substrate for *recomposing* new research from verified parts.

1 INTRODUCTION

The goal of automated AI research is an agent that can take a hypothesis, implement it, run the experiments, and report faithful results. However, evaluating progress toward this goal is difficult precisely because novel research has no ground truth. Paper replication offers an escape: the sub-tasks of replicating a published paper are, to a first approximation, the same sub-tasks required to execute novel research, yet we can use the paper’s reported numbers as a checkable target to verify successful replication.

This makes replication a superb evaluation target, but only if it is scored at the right granularity. Novel research is not a monolith; it decomposes into environment bring-up, data preparation, method implementation, benchmarking, ablation, and analysis—and the vast majority of these sub-tasks replicate sub-tasks that already exist in the literature. Scoring replication as a single binary judgment collapses months of layered work into one bit and hides exactly the structure we want to measure.

PaperBench (Starace et al., 2025) innovated on this in 2024 by explicitly representing the task decomposition of each paper as an expert-authored “rubric” of graded sub-tasks. What is different in 2026 is that these research task decompositions, together with test suites for correctness, can be

generated automatically by terminal agents. We generalize the representation from rubric to **task graph**: a weighted tree of checkable tasks whose section-level nodes additionally carry dependency edges (`depends_on`) recording which task groups must be completed before others can begin—the environment before the method, the method before the experiments that exercise it. The graph turns the binary notion of paper replication into a score: the number of sub-tasks a research agent can successfully implement, verified by executing tests rather than by trusting the agent’s claims.

Automation removes the two constraints that capped decomposition-based benchmarks: expert labor and size. Our generator plans each graph from the paper’s structure and expands every section with its full output budget, so a dense paper receives the decomposition it deserves (228–804 tasks per paper in our corpus; the largest human-authored PaperBench trees run to $\sim 2,000$) rather than what fits in a single model response. The same automation generates the per-task verifiers.

Concretely, we make five contributions:

1. **A public 100-paper corpus with nested splits.** ResearchBench recomposes 100 papers—LLM serving, KV-cache management, quantization, speculative decoding, kernels, compilers, hardware microbenchmarking, and generative optimization—into task graphs, organized into nested splits **ResearchBench-10/-20/-50/-100** by documented criteria (§3), and browsable as interactive graphs at <https://infinity-ai-institute.github.io/research-bench-rubrics/>.
2. **Automatic task-graph generation at unbounded size.** An outline-then-subtree generator with per-section budgets, dependency edges, duplicate-task rejection, and a critique-and-fix convergence pass; graphs reach and exceed human-authored scale without padding (§4).
3. **A layered failure taxonomy** attributing every non-passing task to an owner layer (test harness, result artifact, graph, implementation, environment), separating “the model is weak” from “the harness never asked for the right file” (§5).
4. **Evidence that harness legibility and test-time compute dominate measured agent skill.** On the run-backed 20-paper split, a one-paragraph prompt intervention plus raised compute caps lifts Claude Code from 5.95% to 34.21% implementable-claim closure and Codex from 1.21% to 5.88%, models unchanged (§6).
5. **Behavioral anti-reward-hacking defenses.** Every text-pattern safeguard we deployed was bypassed within one run; defenses that check facts the agent did not author—isolated re-execution with wall-time, GPU-utilization, and file-diff observation—have held (§5.3).

2 RELATED WORK

PaperBench (Starace et al., 2025) introduced expert-authored rubric trees for grading paper replication and remains the reference point for decomposition-based evaluation; our task graphs generalize its representation (dependency edges, machine generation, executable contracts) and our corpus extends its scale by $5\times$. CORE-Bench (Siegel et al., 2024) evaluates computational reproducibility when author code is available; ResearchBench blacklists author repositories to force independent reimplementations. SWE-bench (Jimenez et al., 2024) grades issue resolution against maintainer tests; we similarly grade against executable verifiers, but generate them. MLE-bench (Chan et al., 2024) and RE-Bench (Wijk et al., 2024) measure ML engineering and frontier R&D tasks with curated environments; ResearchBench targets full-paper replication under a declared compute budget. Specification gaming (Krakovna et al., 2020) motivates our behavioral (rather than textual) anti-fabrication defenses.

3 THE RESEARCHBENCH CORPUS AND SPLITS

3.1 PAPERS AND WHY WE CHOSE THEM

The corpus contains 100 papers centered on LLM inference optimization and the systems research surrounding it: serving and scheduling (Orca, PagedAttention, Sarathi-Serve, DistServe, SGLang), KV-cache management and paging, weight and activation quantization (AWQ, SmoothQuant, KIVI, QServe), speculative decoding (EAGLE-3, judge decoding), attention and GPU kernels (FlashAttention 1–3, FlashInfer, FlashDecoding++), compilers and autotuning, hardware microbenchmarking, structured decoding, and LLM-driven generative optimization (AlphaEvolve, FunSearch, kernel-synthesis agents). Papers were sourced from a team-curated list spanning these categories and from a triaged backlog of practitioner-submitted papers, deduplicated against prior sets; each ingested paper is scored for replicability and estimated GPU-hours at triage. The domain is deliberate: inference optimization has hardware-sensitive numbers where “it runs” $\neq$ “it reproduces,” making it the sharpest test bed for faithful replication.

Because the corpus is public and consumers’ hardware differs, the release ships the *task-breakdown graphs with dependency structure only*: we do not bake our own hardware’s feasibility judgments into the public artifact. Per-task preconditions (required hardware, datasets, pretrained models) are recorded so any consumer can compute their own achievable subset.

3.2 NESTED SPLITS: RESEARCHBENCH-10/-20/-50/-100

The corpus is organized into four nested splits, each a subset of the next, assigned by a documented, mechanical criterion (published with the corpus): papers are ranked by *run maturity* (papers with completed clean-baseline agent runs rank first—their scores are ground truth), then *triage replicability*, then *inverse compute burden* (estimated GPU-hours), with a greedy category-coverage adjustment at each tier boundary so that no split collapses into a single topic.

- **ResearchBench-10**: the ground-truthed core—papers with completed baseline runs, single-GPU feasible, spanning the major categories. The split to cite for verified, comparable agent scores.
- **ResearchBench-20**: the full run-backed evaluation split used in §6 (frozen 2026-06-25; 7,290 tasks; 1,327 implementable headline claims; every task’s contract content-hashed).
- **ResearchBench-50**: adds category breadth—quantization, speculative decoding, kernels, compilers—papers with gated task graphs that have not yet been run end-to-end.
- **ResearchBench-100**: the full corpus, including heavy/multi-GPU and exotic-hardware papers where the graph itself is the contribution and replication awaits matching compute.

All splits are browsable at <https://infinity-ai-institute.github.io/research-bench-rubrics/>: the site sorts papers into the four splits, and clicking any paper opens its interactive task graph (collapsible D3 tree with per-task category coloring, verification specs, and dependency metadata); split members whose graphs are still generating are listed as pending. **[TODO: finalize -50/-100 membership when corpus generation completes]**

3.3 REPLICABLE VS. UNREPLICABLE PAPERS

Unreplicability is recorded per task, not per paper. Tasks whose preconditions cannot be met in a declared environment (hardware the original authors used but the consumer lacks; heavyweight pretrained models; proprietary traces) are classified *skipped-environment* and excluded from that environment’s achievable denominator, with audited skip reasons. On our reference H100, three papers of the run-backed split have no implementable headline baseline and score n/a; the remaining 17 are scoreable end-to-end. As an example of the environment split, one KV-cache paper decomposes into 309 tasks of which 218 are environment-skipped and 91 achievable.

3.4 COMPUTE BURDEN AND REPLICABILITY

Replicability and compute burden are orthogonal axes, and both are scored at triage: a paper can be perfectly replicable but need 500 GPU-hours, or cheap but blocked on a proprietary artifact. The corpus spans this plane deliberately—microbenchmarking papers replicate in minutes; serving papers need hours of load generation; quantization papers need model downloads but little compute; kernel papers need specific GPU generations. Split placement uses estimated GPU-hours so the lower splits stay affordable: the -10 split is runnable end-to-end for under \$100 of GPU time per agent, while -100 admits papers whose full replication exceeds any single-node budget.

3.5 DECIDING WHAT TO REPLICATE GIVEN RESOURCES

Each evaluation run gives the agent an explicit resource contract: a dollar budget, a wall-clock cap, a maximum number of iterations, and the paper’s feasibility estimate. Agents are instructed to scope honestly—documenting skipped tasks rather than fabricating them—and, per iteration, are focused on a small *target slice* of open tasks with exact expected outputs, which converts a several-hundred-task graph into tractable turns. The dependency edges give the agent (and the harness’s slice selector) the order in which task groups unlock: environment and data tasks first, method tasks next, measured experiment tasks last.

4 TASK GRAPHS: STRUCTURE, GENERATION, AND VERIFICATION

4.1 AUTOMATIC GENERATION AT UNBOUNDED SIZE

For each paper, a pipeline of terminal-agent stages produces the graph without expert authorship: ingestion converts the arXiv source to markdown; a triage stage scores replicability and GPU-hours; a claim extractor records the paper’s numerical results; and the graph generator runs in two stages. An *outline* call plans the top levels of the graph from the paper’s structure—one section per method component, experiment or table family, dataset group, and ablation study—assigning each section a task budget scaled by its claim and table density, the numerical claims it must cover, and its dependency edges. Each section is then expanded by its own *subtree* call with the full model output budget to itself and a digest of sibling sections to suppress cross-section duplicates. This outline-then-subtree design removes the size ceiling that a single-response generation imposes: no paper’s decomposition is compressed to fit one model response. A critique-and-fix pass then converges the graph (materializing missing coverage in bounded batches, annotating verification metadata, salvaging schema violations), and a deterministic gate rejects graphs with duplicate tasks—size must come from real decomposition, not repetition.

4.2 GRAPH STRUCTURE: NUMBER OF TASKS AND GRANULARITY

Graphs in the corpus run from ~ 100 to ~ 935 tasks per paper (median ~ 300), at depths of 3–7. Granularity follows a code-exists $\rightarrow$ code-behaves $\rightarrow$ result-matches progression: each method component decomposes into structural tasks (the API surface exists), behavioral tasks (deterministic computation matches), and measured tasks (a paper number is independently reproduced within tolerance). Single-response generation systematically under-decomposes: on papers with human-authored PaperBench references, our earlier capped generator produced graphs at a median $0.51\times$ the human size (worst case $0.04\times$); the outline-then-subtree generator reaches a median $1.2\times$ with zero duplicate-task padding, and per-paper extremes moved from 40-vs-997 tasks to 300-vs-997 on the densest paper. Granularity is audited by the gate: tasks that merely restate their parent, or that split one check into trivial variants, are rejected.

4.3 VERIFICATION CONTRACTS

Every measured task is bound to an executable contract specifying the exact artifact path the implementation must emit and the tolerance under which the measured value is accepted, e.g.:

Table 1: Failure taxonomy. Every non-passing task is attributed to a category and owner layer.

Category	Owner layer	Meaning
missing_per_leaf_test	test harness	no executable verifier was produced for the task
missing_result_artifact	result artifact	the contract-named output file was never emitted
broad_leaf_unverifiable	graph	task too coarse to verify; needs splitting
verification_gap	implementation	code ran; evidence does not match the contract
numeric_mismatch	implementation	measured, but outside tolerance
environment_blocked	environment	hardware/model/dataset unavailable

```
expected_output = results/leaf.<id>.json; tolerance: abs(measured -
2.1)/2.1 ≤ 0.4
```

A task is credited only when a pytest verifier at a deterministic path (`_tests_local/test_leaf-<id>.py`) passes while reading that artifact. Credit is attributed by executing tests, never by the agent’s claims.

5 EVALUATION METHODOLOGY

5.1 AGENT CONTRACT AND ITERATED SESSIONS

The replication agent receives only `paper.md`, the interface contract, the task graph, the verification contracts, and stock terminal tools (read, write, edit, bash, glob, grep). The paper’s official repository is blacklisted, forcing independent reimplementations. Because agent sessions cannot be resumed, all state lives on disk (progress log, source tree, per-task tests, results); an outer loop re-invokes the agent in fresh sessions—selecting a repair slice, running a turn, grading, attributing failures, and choosing the next slice—until headline closure validates or budget/iteration/wall caps are reached. A full 20-paper backend run completes in roughly 10–14 hours on one H100.

5.2 GRADING AND FAILURE ATTRIBUTION

Grading is three-layered: per-task test attribution (which executable verifier passed), reproduction quality (§5.3), and aggregate scores. Every non-passing task is assigned a failure category and an *owner layer* (Table 1). This attribution is what makes harness optimization tractable: without it, harness illegibility is indistinguishable from model weakness and benchmarks will mis-rank models.

5.3 BEHAVIORAL DEFENSES AGAINST REWARD HACKING

Agents under budget pressure fabricate: hard-coded “results,” tests that assert paper constants, and confident reports of runs that never happened. Our experience is that *every* text-pattern safeguard was bypassed within one run of deployment: a regex confession audit for terms like “cached”/“surrogate” was defeated by renaming to `paper_reported`; an empty-source negative control was defeated by tests that passed on stubs; a “do not fabricate” prompt directive had no measurable effect.

Durable safeguards check facts about the run that the agent did not author. The primary mechanism is **isolated reproduction**: the grader deletes the agent’s `results/` directory, re-runs the agent’s `reproduce.sh` from scratch, and observes wall time, GPU utilization, and file diffs. Runs whose regenerated results are byte-identical re-ships of pre-baked constants, or that complete implausibly fast with no GPU activity, are classified *suspect* and their scores gated (Appendix C). Planned extensions include cost/time floors (actual spend versus the claimed-tasks estimate) and tool-use forensics on agent transcripts. Our design rule: *if a safeguard can be defeated by different words, it will be*.

Table 2: Where implementable work remained open (both backends, Attempt 2).

Missed task type	Unclosed tasks
Evaluation, Metrics & Benchmarking	1448
Method Implementation	955
Experimental Setup	450
Environment & Infrastructure Setup	47
Dataset and Model Acquisition	43
Logging, Analysis & Presentation	28
Data Processing & Preparation	27

6 EXPERIMENTS: SUCCESS RATES ON RESEARCHBENCH-20

6.1 SETUP: SAME-SPLIT ATTEMPTS

All experiments run on the frozen ResearchBench-20 split with byte-identical graphs and contracts, one shared controller, and only the backend adapter varying between Claude Code (`claude-opus-4-7`) and Codex (`gpt-5-codex`). Attempt 1 established the shared controller on a mixed-source snapshot. Attempt 2 is the **clean baseline**: stock harness, prompts, and adapters, run from a clean tree at committed `main` (budget \$200/paper, 15 iterations, 8h wall). Attempt 3 applies a single **harness intervention** (below) with raised compute caps (\$1000/paper, 40 iterations, 12h wall).

6.2 SUCCESS RATES OF THE AGENTS (CLEAN BASELINE)

Codex closed 16/1327 implementable claims (1.21%) and Claude Code 79/1327 (5.95%); average total-implemented across all graph tasks was 0.21% and 1.44% respectively. Claude led on 12 papers, Codex on 2, with 3 tied. The two backends failed differently: Codex predominantly stopped before producing artifacts (13/20 papers with replication runtime failures; 1,231 tasks blocked on `missing_result_artifact`), while Claude more often reached verification and stalled on coverage (13/20 papers ending `reproduction_not_verified_within_budget`; 1,233 tasks on `missing_per_leaf_test`; 73 on `numeric_mismatch`—Claude got to measured numbers more often, and sometimes missed tolerance).

6.3 PERFORMANCE ACROSS TASK CATEGORIES

Unclosed implementable work concentrates in benchmark-heavy categories (Table 2): running and measuring, not code-writing, is where agents fail. Structural and behavioral tasks (does the component exist; does it compute correctly on deterministic inputs) close at several times the rate of measured tasks (does the full experiment reproduce the paper’s number), and within measured tasks, those requiring long-running load generation or model downloads close least. This gradient replicates across both backends and both graph provenances we have evaluated, suggesting it reflects genuine difficulty ordering rather than graph idiosyncrasy.

6.4 DIAGNOSIS: HARNESS LEGIBILITY, NOT MODEL CAPABILITY

Failure attribution located the bottleneck precisely. A task can only be credited when a `pytest` file exists at the deterministic path `_tests_local/test_leaf-<id>.py` and passes while reading the contract-named artifact. Inspection of failed workdirs showed agents writing correct-*shaped* tests and artifacts, but only for the current target slice (~4 tasks), leaving 80+ achievable tasks per paper unverified: the implementation existed, but the grader was never shown the files it checks. Nothing in the baseline prompts stated the task→file mapping or instructed exhaustive coverage.

Table 3: Implementable-claim closure across same-split attempts. Models unchanged throughout.

Attempt	Change vs. prior	Codex	Claude Code
2 (clean baseline)	none	16/1327 = 1.21%	79/1327 = 5.95%
3 (legibility patch)	prompt paragraph + raised caps	78/1327 = 5.88%	454/1327 = 34.21%

6.5 IMPACT OF THE INTERVENTION AND ADDITIONAL TEST-TIME COMPUTE

We appended one paragraph to each backend’s prompt stating (i) the deterministic task→test-path and contract→artifact mapping, (ii) an instruction to enumerate *every* achievable task rather than the current slice, and (iii) a reminder that `reproduce.sh` must regenerate `results/` from scratch (Appendix B); and we raised the compute caps from \$200/15 iterations/8h to \$1000/40 iterations/12h. No frozen artifact, grader, or adapter changed; hash checks confirm identical graphs and contracts.

Both backends improved roughly fivefold (Table 3): Codex’s closure rose $4.9\times$ (average total-implemented 0.21% to 1.63%) and Claude Code’s $5.7\times$ (1.44% to 8.96%), with five papers exceeding 70% closure under Claude—including a historic stuck case that had scored zero in every prior run. Additional test-time compute is a genuine and separable lever: in the baseline, most Claude papers ended `blocked` well under their caps (the loop stalled before spending its budget), whereas under the intervention agents productively consume the raised iteration budget covering hundreds of tasks per paper. But compute alone could not have produced files the agent did not know to write—and the intervention alone, under starved caps, could not cover 300-task graphs. We emphasize the confound: Attempt 3 changed the prompt *and* the caps, and ran both backends concurrently on one H100 (a minor contention risk for latency-sensitive measured tasks). Isolating the levers is Attempt 4. Nevertheless, the qualitative conclusion is robust: a large fraction of what a naive benchmark would report as “agent failure” was *harness legibility* failure, and much of the rest was starvation of test-time compute.

6.6 HARNESS EVALUATION INDEPENDENT OF THE MODEL

Because inputs, graphs, and contracts are frozen and only the backend adapter varies, cross-backend differences attribute to the model-plus-harness pair, and attempt-over-attempt deltas attribute to the harness alone. We propose a baseline grid crossing models (Claude, Codex, GLM 5.2) with harnesses (OpenCode, Cursor, Claude Code); the Claude Code harness column is measured in this paper and the remaining cells are planned. [TODO: fill grid cells as runs complete]

7 DISCUSSION AND FUTURE WORK

Recomposition: from replication to novel research. The premise of this benchmark is that papers break down into the same ideas novel research is built from. Task graphs make that premise operational: a library of 100 graphs is a library of verified, reusable research sub-tasks—environment recipes, method implementations, measurement harnesses—each with an executable correctness contract. A new idea can then be expressed as a novel *composition* of existing sub-tasks (one paper’s KV-cache tiering with another’s prefill scheduling, evaluated under a third’s load harness), graded with the same contracts and isolated reproduction that grade replication. This extends the benchmark from replication toward automated research execution, including tasks proposed by AI hypothesis systems: the same graph machinery that scores an agent’s replication can scaffold, and audit, its first steps beyond the paper.

Automated harness optimization. The failure taxonomy is machine-readable (per-run category and owner-layer counts). The Attempt-3 fix was found by reading that attribution; the loop can be closed automatically: run → attribute failures → propose a harness patch → smoke on one paper → full run → compare on the frozen split. Frozen hashes make every candidate comparable, turning

the benchmark into a reward signal for harness search. A guardrail from our experience: optimize against a diverse rotation of paper types, never a single stuck paper.

Production integration. The motivating use case is a company that wants a relevant paper’s improvements in its internal, performance-sensitive codebase—today, manual researcher work. The same contracts that grade replication become acceptance tests for landing the technique in production CI.

Limitations. (i) Graph quality is itself machine-generated; the `broadleaf_unverifiable` class quantifies, but does not eliminate, coarse tasks, and per-paper scores are sensitive to graph provenance even when aggregates agree. (ii) Attempt-3 confounds prompt and compute changes (isolation planned). (iii) Run-backed evaluation currently covers the inference-systems splits on one hardware target; the -50/-100 splits ship graphs ahead of runs. (iv) Two backends shared a GPU in Attempt 3, a contention risk for a small number of latency-sensitive tasks.

8 CONCLUSION

ResearchBench recomposes 100 research papers into automatically generated, dependency-structured task graphs with executable verification, organized into nested splits and browsable publicly, and demonstrates the full loop on its run-backed split: budgeted agent execution, per-task grading, failure attribution, and anti-fabrication auditing. Its first empirical lesson is cautionary for the field: the largest levers on measured “agent research ability” were not the model but whether the harness made the grading contract legible and whether the agent was given enough test-time compute to cover the graph. Its design bet is constructive: if papers decompose into the sub-tasks novel research composes, then a public library of verified task graphs is not just a benchmark—it is a parts inventory for automated research. We release the corpus, splits, site, and attempt history as a standing scoreboard and invite the community to measure models, harnesses, and compute policies against it.

REFERENCES

- J. S. Chan, N. Chowdhury, O. Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv:2410.07095*, 2024.
- C. E. Jimenez, J. Yang, A. Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.
- V. Krakovna, J. Uesato, V. Mikulik, et al. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.
- Z. S. Siegel, S. Kapoor, N. Nadgir, B. Stroebel, and A. Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv:2409.11363*, 2024.
- G. Starace, O. Jaffe, D. Sherburn, et al. PaperBench: Evaluating AI’s ability to replicate AI research. *arXiv:2504.01848*, 2025.
- H. Wijk, T. Lin, J. Becker, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv:2411.15114*, 2024.

A RESEARCHBENCH-20 COMPOSITION

B THE ATTEMPT-3 LEGIBILITY PARAGRAPH

The full text appended to both backends’ prompts (verbatim):

Table 4: Papers in the run-backed ResearchBench-20 split with task counts. Three papers (marked †) have no implementable headline baseline in the reference environment and score n/a. The full corpus roster with split membership is maintained on the project site (§3.2).

Paper (slug, abbreviated)	Tasks	Paper	Tasks
llmservingsim-2.0	214	icecache	935
FOCUS-autokernel	268	lamp	288
fp16-equivalence	650	niyama	160
nexus	312	prefill-as-a-service	532
adaserve	692	ragged-paged-attention	442
asynctl	475	shift-parallelism	407
cacheflow	177	ttkv	565
contiguouskv	309	tutti	220
duetserve	350	fast-heterogeneous†	93
flexicache†	96	hexgen-flow†	105
Total			7,290

LEAF -> FILE PATH MAPPING (this is how you score; read carefully):

- A rubric leaf is counted as verified ONLY when a pytest file exists at the exact path `.tests.local/test.leaf-<LEAF.ID>.py` and passes, where `<LEAF.ID>` is the leaf's id with every hyphen '-' replaced by '_'. If that file is missing, the leaf stays unverified and scores zero no matter how correct your src/ code is.
- For measured / result.match leaves, that test must load the artifact named by the leaf's contract expected.output (see `../verification.contracts.json`; convention `results/leaf-<LEAF.ID>.json`) and assert the contract's field(s)/tolerance.
- Enumerate EVERY leaf in `../rubric.json` and `../verification.contracts.json`. For each leaf NOT blocked by unavailable hardware/models/datasets (i.e. every achievable leaf, not just the current target slice), write its `.tests.local/test.leaf-<LEAF.ID>.py` AND emit the result artifact that test reads. Breadth of achievable-leaf coverage is the single biggest lever on your score.
- `reproduce.sh` must regenerate all of `results/` from scratch (the grader wipes `results/` and reruns `reproduce.sh` in isolation).

C REPRODUCTION QUALITY GATES

Isolated reproduction classifies each run as ok, suspect, or failed: failed if `reproduce.sh` is absent, exits nonzero, times out, or produces zero files into a wiped `results/`; suspect if wall time is implausibly short with no GPU activity, or if more than half of the regenerated files are byte-identical to the agent's pre-authored results with no authored code paths present. Suspect and failed runs have their task credit gated.